Respostas Lab1 SOSIM

Exercício 1: Escalonamento circular

- Configurar o escalonamento circular (sem prioridade): janela *Gerência do Processador* / *Opções*
- Criar dois processos com a mesma prioridade (um CPU-bound e outro I/O-bound)

Na janela *Console Sosim* / *Processos* / *Selecionar* observe o tempo de processador de cada processo durante três minutos e as mudanças de estado. Após esse período anote o tempo de processador de cada processo. Analise a distribuição do uso da UCP.

Análise da distribuição do uso do UCP:

Após o tempo de três minutos de processamento, obtive resultados que o tempo total de CPU do CPU-bound foi de 84, enquanto o tempo total de CPU do I/O-bound foi de 17. Há uma grande disparidade a favor do processo CPU-bound (cerca de 5x mais uso de UCP). Isso acontece porque o CPU-bound permanece constantemente nos estados Executando e Pronto, aproveitando até o fim cada fatia de tempo concedida. Já o processo I/O-bound abdica voluntariamente da UCP em poucos milissegundos para realizar operações de E/S, passando a maior parte do tempo bloqueado no estado de Espera.

Pergunta: O que acontece se o tempo de time slice (quantum) aumentar ou diminuir?

Resposta:

Se o quantum aumentar: O processo CPU-bound passará períodos mais longos e contínuos na UCP, reduzindo a troca de contexto no sistema. Em contrapartida, o tempo de resposta do I/O-bound piora, pois quando ele sai da espera, precisa aguardar um quantum maior do CPU-bound terminar para conseguir rodar. Se o quantum diminuir: Aumenta a frequência de trocas de contexto e o overhead do SO. O tempo de resposta para o I/O-bound melhora (pois o CPU-bound é interrompido mais cedo), mas o tempo total acumulado de UCP do I/O-bound continuará muito menor, pois ele ainda se bloqueará por E/S antes mesmo do quantum finalizar.

Exercício 2: Escalonamento circular com prioridades

- Configurar o escalonamento por prioridades: janela *Gerência do Processador* / *Opções*
- Criar um processo CPU-bound com prioridade 3 e um I/O-bound com prioridade 4.

Na janela *Console Sosim* / *Processos* / *Selecionar* observe o tempo de processador de cada processo durante três minutos e as mudanças de estado. Após esse período anote o tempo de processador de cada processo. Analise a distribuição do uso da UCP comparativamente ao Exercício 1.

Análise comparativa ao Exercício 1:

Após o tempo de três minutos de processamento, obtive resultados que o tempo total de CPU do CPU-bound (Prioridade 3) foi de 81, enquanto o tempo total de CPU do I/O-bound (Prioridade 4) foi de 17. Assim, conclui-se que atribuir uma prioridade maior ao I/O-bound permitiu que ele tirasse levemente o CPU-bound da UCP assim que retornava do estado de espera (reduzindo a UCP do CPU-bound de 84 para 81). No entanto, o tempo acumulado pelo I/O-bound permaneceu exatamente o mesmo (17), demonstrando que o seu gargalo principal é o tempo que passa bloqueado em E/S, e não o algoritmo de escalonamento.

Pergunta: O que acontece se o tempo de espera do processo I/O-bound aumentar ou diminuir?

Resposta:

Se o tempo de espera aumentar, o processo I/O-bound passará mais tempo bloqueado na fila de E/S. Consequentemente, ele usará ainda menos a UCP, aumentando a diferença a favor do processo CPU-bound. Se o tempo de espera diminuir, o processo I/O-bound retornará da E/S mais rapidamente e, como possui prioridade maior (4), interrompe o CPU-bound com mais frequência. Isso reduzirá ligeiramente o tempo acumulado do CPU-bound e aumentará o uso de UCP do I/O-bound.

Exercício 3: Escalonamento circular com prioridades

- Configurar o escalonamento por prioridades: janela *Gerência do Processador* / *Opções*
- Criar um processo CPU-bound com prioridade 4 e um I/O-bound com prioridade 3.

Analise a situação.

Comportamento: Agora o processo CPU-bound tem prioridade maior (4) e o I/O-bound tem prioridade menor (3). Como o CPU-bound nunca libera a UCP por E/S e tem prioridade maior, o escalonador dará preferência absoluta a ele na fila de Pronto. O processo I/O-bound sofrerá de starvation (privação de CPU). Mesmo quando ele concluir uma operação de E/S e voltar para o estado de Pronto, ele não conseguirá executar enquanto o CPU-bound estiver na fila, pois a prioridade 3 é inferior. O tempo de UCP acumulado pelo I/O-bound será mínimo ou praticamente zero.

Pergunta: Quais devem ser os critérios para determinar as prioridades de processos?

Resposta:

Para garantir eficiência e justiça no sistema operacional, as prioridades devem ser atribuídas com base nos seguintes critérios:

Processos I/O-bound devem ter maior prioridade. Como usam a UCP por pouquíssimo tempo antes de se bloquearem para E/S, dar prioridade a eles mantém

os periféricos/dispositivos ocupados sem prejudicar o tempo de CPU do processo CPU-bound. Processos interativos (com interface com o usuário) exigem prioridade mais alta para evitar congelamentos da tela e garantir respostas rápidas aos comandos do usuário.O SO deve adotar prioridades dinâmicas (agentes que aumentam temporariamente a prioridade de um processo que está há muito tempo esperando na fila de Pronto). Processos de sistema (kernel, drivers, tarefas em tempo real) devem ter prioridade superior aos processos comuns de usuário.

Exercício 4: Escalonamento circular com prioridades (prioridade dinâmica – mecanismo adaptativo)

- Configurar o escalonamento por prioridades: janela *Gerência do Processador* / *Opções*
- Criar dois processos com a mesma prioridade (um CPU-bound e outro I/O-bound)

Observe o escalonamento dos processos. Compare a situação desse escalonamento com o apresentado no Exercício 2.

Pergunta: Qual a vantagem desse escalonamento em processos I/O-bound de perfis diferentes?

Análise do escalonamento:

Comportamento observado: Com o mecanismo adaptativo (prioridade dinâmica) ativado no SOsim, o próprio sistema operacional ajusta as prioridades durante a execução. Quando um processo abre mão da UCP para realizar I/O, o SO aumenta temporariamente sua prioridade (+2,). Por outro lado, processos que consomem todo o quantum (CPU-bound) têm sua prioridade reduzida ou mantida baixa. No Exercício 2, as prioridades eram estáticas (definidas manualmente no início). No Exercício 4, o SO identifica o comportamento dos processos automaticamente. O I/O-bound ganha prioridade dinâmica superior sem precisar de intervenção manual, garantindo que ele passe à frente do CPU-bound na fila de Pronto sempre que retornar do bloqueio de E/S.

Pergunta: Qual a vantagem desse escalonamento em processos I/O-bound de perfis diferentes?

Resposta:

Ajuste Automático e Justo: O sistema identifica sozinho quais processos dependem mais de E/S (como interfaces gráficas ou requisições de rede) sem que o usuário ou programador precise definir prioridades manuais. Processos I/O-bound que interagem com o usuário ou que realizam operações rápidas ganham prioridade dinamicamente assim que saem do estado de espera, respondendo quase que imediatamente. Como o SO "repõe" a prioridade do I/O-bound rapidamente, os dispositivos de entrada e saída permanecem sempre ocupados, aumentando a eficiência geral do sistema (*throughput*). Processos I/O-bound com taxas de E/S muito distintas recebem aumentos de prioridade proporcionais ao tempo de espera, evitando que processos mais lentos fiquem esquecidos na fila enquanto um CPU-bound monopoliza o processador.